— PORTFOLIO · 2026

Sean Kakhkhorov

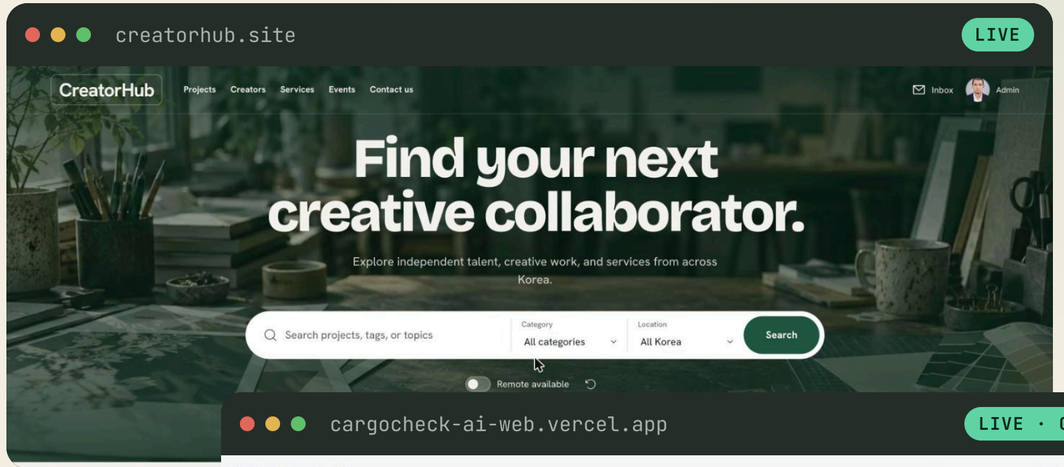
Full Stack Developer • **Agentic Engineer** • **AI Engineer** • **DevOps**

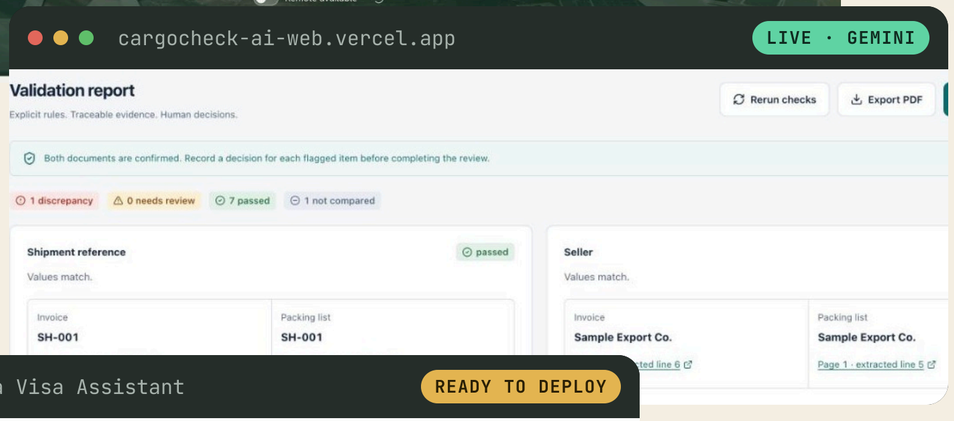
I build web products end to end and keep them running: **three live products** and two more ready to deploy, from a creator marketplace to AI document review and cited legal search.

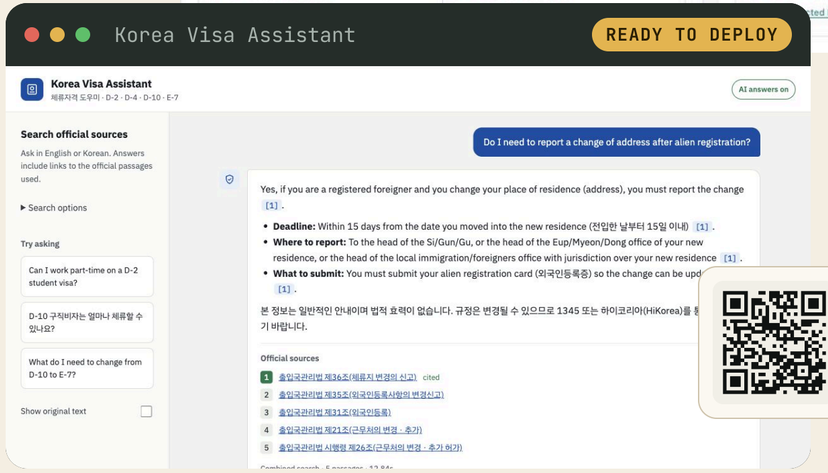
3+ years of professional web development	**20+** client companies served at IT Local	**3** live products: 2 solo, 1 collaborative	**293** automated checks recorded in CreatorHub

creatorhub.site · cargocheck-ai-web.vercel.app · sean-kakhkhorov.site

Suwon, South Korea · kakhkhorov97@gmail.com · github.com/kakhkhorov97





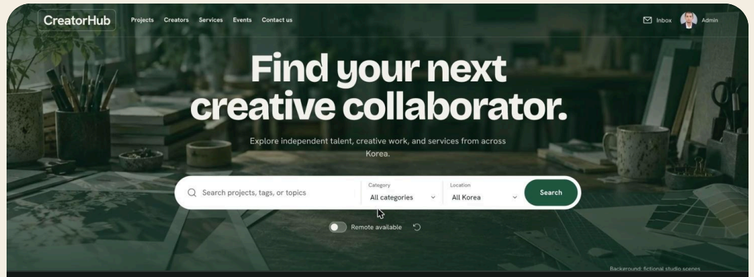


Full portfolio
sean-kakhkhorov.site

— PROJECTS AT A GLANCE

What I have built.

Also inside: About p. 03 · Experience p. 04 · Skills p. 06 · Contact p. 27

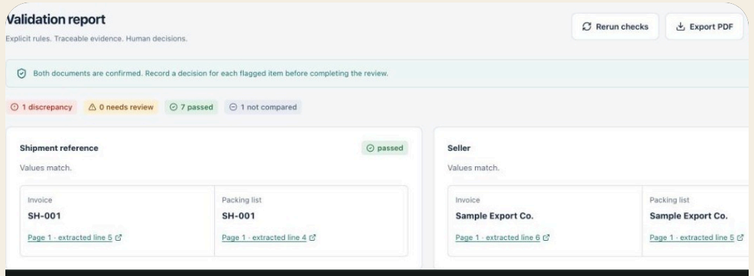


CreatorHub LIVE · SOLO

Creator marketplace for Korea with private chat, moderation and a Gemini assistant.

Next.js · NestJS · GraphQL · MongoDB

creatorhub.site ↗ p. 07 →

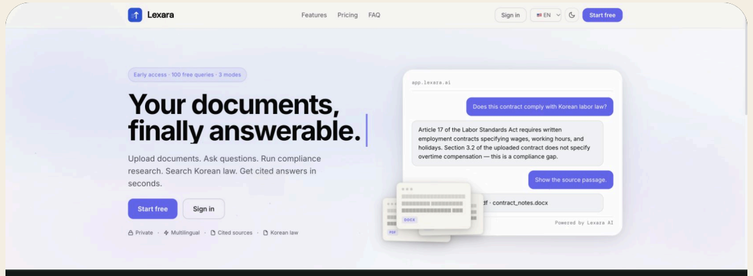


CargoCheck AI LIVE · GEMINI

AI document review: Gemini extraction, human confirmation, explicit rule checks.

React · NestJS · Neon · Upstash

cargocheck-ai-web.vercel.app ↗ p. 17 →

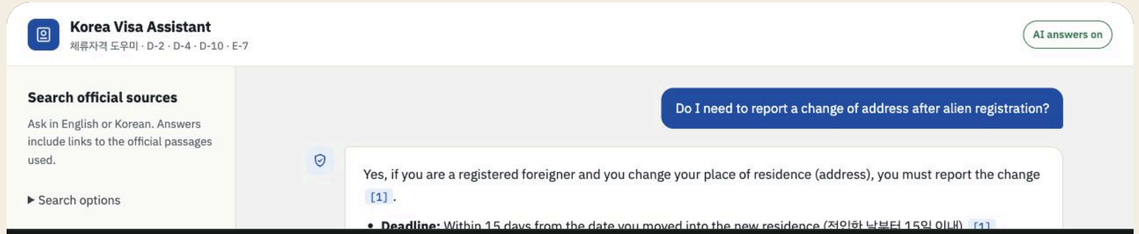


Lexara LIVE · COLLABORATIVE

Document Q&A with cited answers. I contributed as one of the developers.

FastAPI · PostgreSQL · React

lexara-top.vercel.app ↗ p. 23 →

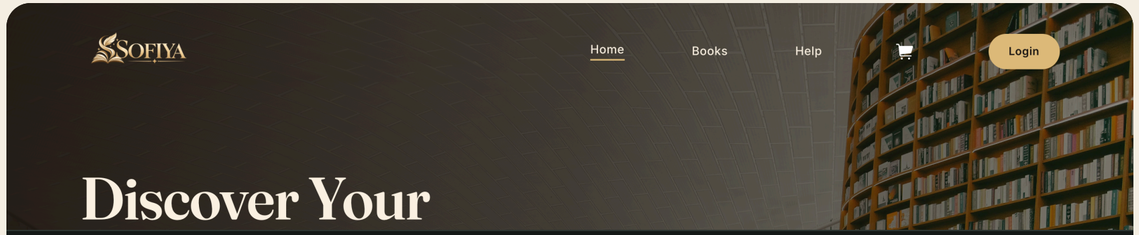


Korea Visa Assistant READY TO DEPLOY

Cited answers over Korean immigration law; hybrid retrieval hit@5 1.00 on 11 questions.

Python · FastAPI · Chroma · Gemini

GitHub ↗ p. 24 →



Sofiya Bookstore READY TO DEPLOY

Bookstore with basket, order tracking and an EJS admin panel.

React · Redux Toolkit · Express · MongoDB

GitHub ↗ p. 26 →

— ABOUT ME

Full-stack development, from code to production.

I started as a backend developer building Express and MongoDB APIs, then worked as a full-stack developer at IT Local from June 2023 to January 2026, in a team of four or more developers serving more than 20 client companies. There I looked after everything from database design and authentication to deployments, DNS and outages, which taught me to care about how code behaves after release, not just how it is written.

Since then I have shipped my own products: **CreatorHub**, a creator marketplace I built and run alone at **creatorhub.site**; **CargoCheck AI**, an AI document-review app live at **cargocheck-ai-web.vercel.app**; and **Lexara**, a document Q&A platform I helped another developer build. **Korea Visa Assistant** and **Sofiya Bookstore** are ready to deploy.

I use coding agents such as Claude Code and Codex within written repository rules: I approve each change, and automated checks run before anything is merged.

Reliability
Monitoring, logs and quick outage fixes

Security
Auth, input validation and code review

Documentation
APIs and deploy steps written down

AI workflow
Coding agents with review gates



Kakhkhorov Shokhjakhon (Sean)
Full Stack Developer · Agentic Engineer
AI Engineer · DevOps

— EXPERIENCE

Full-stack Web Developer
IT Local · 2023.06 – 2026.01

Backend Developer
Atlantis Investment · 2022.05 – 2022.12

— BASED IN

Suwon, Gyeonggi-do, South Korea

— FIND ME

github.com/kakhkhorov97
kakhkhorov97@gmail.com

— EXPERIENCE

Where I have worked.

Full-stack Web Developer

2023.06 - 2026.01

IT Local · Uzbekistan

IT services company offering web development, server and computer assembly, network configuration and security-camera installation.

- Built and maintained websites for 20+ client companies in construction, retail, food service, corporate and public sectors, in a team of 4+ developers
- Developed APIs and server-side business logic with authentication, authorization and input validation
- Designed database schemas; improved page and API response times through indexing and query optimization
- Set up and secured 5–15 Ubuntu servers, including SSH access and firewall rules
- Ran Node.js services with PM2 and Docker behind Nginx, using Cloudflare for DNS and TLS
- Moved deployments from manual steps to CI-based automated deployment
- Monitored application health, analyzed logs and resolved outages and performance issues
- Reviewed teammates' code; documented APIs and deployment procedures
- Built responsive pages with React, Bootstrap and Tailwind CSS
- Onboarded my replacement and handed over every system before leaving

Backend Developer

2022.05 - 2022.12

Atlantis Investment · Uzbekistan

Alongside my university studies.

- Built RESTful APIs with Express.js between the frontend and backend
- Designed and maintained MongoDB databases, using Mongoose for data modeling and access
- Applied MVC architecture to separate concerns and keep the code easy to maintain and scale
- Implemented sign-up and login with sessions and JWT, hashing passwords with bcryptjs
- Built file upload endpoints with Multer for images and documents
- Validated request data and added consistent error handling and status codes
- Wrote MongoDB queries and indexes for search, filtering and pagination
- Tested and documented endpoints in Postman and agreed request and response formats with the frontend

— EXPERIENCE · IT LOCAL

Client websites, from build to servers.

IT Local builds and runs websites for other businesses. In a team of four or more developers I worked across the whole path, from the first page to the server it runs on.

20+ client companies
Construction, retail, food service, corporate and public sector

→

Team of 4+ developers
Backend, frontend, servers and code review

→

CI deployment
Replaced manual deploy steps

→

5–15 Ubuntu servers
Nginx, PM2, Docker and Cloudflare

— THREE PROBLEMS I SOLVED

Manual deploys → CI

Problem. Releases were done by hand: SSH into each server, pull the code, install and restart. Across many client sites on 5–15 servers, a missed step could break a site, and only people who knew the steps could deploy.

What I did. Wrote the steps down, then moved them into a CI pipeline that installs, builds and restarts the service the same way every time.

Result. Every deploy followed the same reviewed steps, and other developers could release without me.

A client site goes down

Problem. Client sites occasionally went down or started returning errors behind Nginx.

What I did. Traced each incident through the Nginx, PM2 and application logs to the point of failure, fixed the underlying issue, and set PM2 to restart services automatically.

Result. The sites were brought back, and each fix was written down so the team could follow the same steps next time.

Slow pages as data grew

Problem. List and search pages got slower as client data grew.

What I did. Checked the slow queries, added indexes that match the filters and sort order, and paginated results instead of loading everything.

Result. Queries used the new indexes, so list and search pages stayed responsive as data kept growing. I did not record before-and-after timings.

Client sites are not shown here: they belong to IT Local's customers.

— SKILLS & TOOLS

My stack.

CORE, USED DAILY ALSO USED

Frontend

React · Next.js · TypeScript · JavaScript · Apollo Client · Redux Toolkit · MUI · Tailwind CSS · SCSS · HTML · CSS · jQuery · Axios · TanStack Query · React Router · Socket.IO Client · Framer Motion · Three.js · Swiper · Anime.js · Typed.js · TUI Editor · Bootstrap · Chakra UI · shadcn/ui · Figma

Backend

Node.js · NestJS · Express · GraphQL · REST APIs · Socket.io · JWT & OAuth · Python · FastAPI · Zod · Role-based access · EJS · bcryptjs · Multer

Databases

MongoDB · Mongoose · PostgreSQL (Neon) · Redis (Upstash) · Transactions · Indexing & query tuning · BullMQ · Chroma · MySQL · MongoDB Compass

DevOps & cloud

Docker · Docker Compose · Nginx · GitHub Actions · Vercel · Render · Cloudflare · Linux (Ubuntu) · DNS & TLS · PM2 · Firewall configuration · Git & GitHub

AI & retrieval

Gemini API · RAG: BM25 + vectors (RRF) · Structured extraction · OCR (Tesseract, Poppler) · OpenAI API · Kiwi tokenizer · Chroma · hit@k · recall@k · MRR · Claude Code · Codex · Cursor

Testing & tools

Jest · Supertest · ESLint · Prettier · pytest · Integration tests · API contract tests · PostHog · Postman · npm

Where they run: Next.js, NestJS, GraphQL, MongoDB, Docker and Nginx in CreatorHub (live) · React, NestJS, Gemini, Neon PostgreSQL, Upstash Redis, Vercel and Render in CargoCheck AI (live) · Python, FastAPI, Kiwi, BM25, Chroma and Gemini in Korea Visa Assistant · FastAPI and pgvector in Lexara (collaborative).

— SELECTED PROJECT · 01 · LIVE

CreatorHub

Solo project, 2026.04 – present: I designed, built and deployed the frontend, backend and infrastructure.

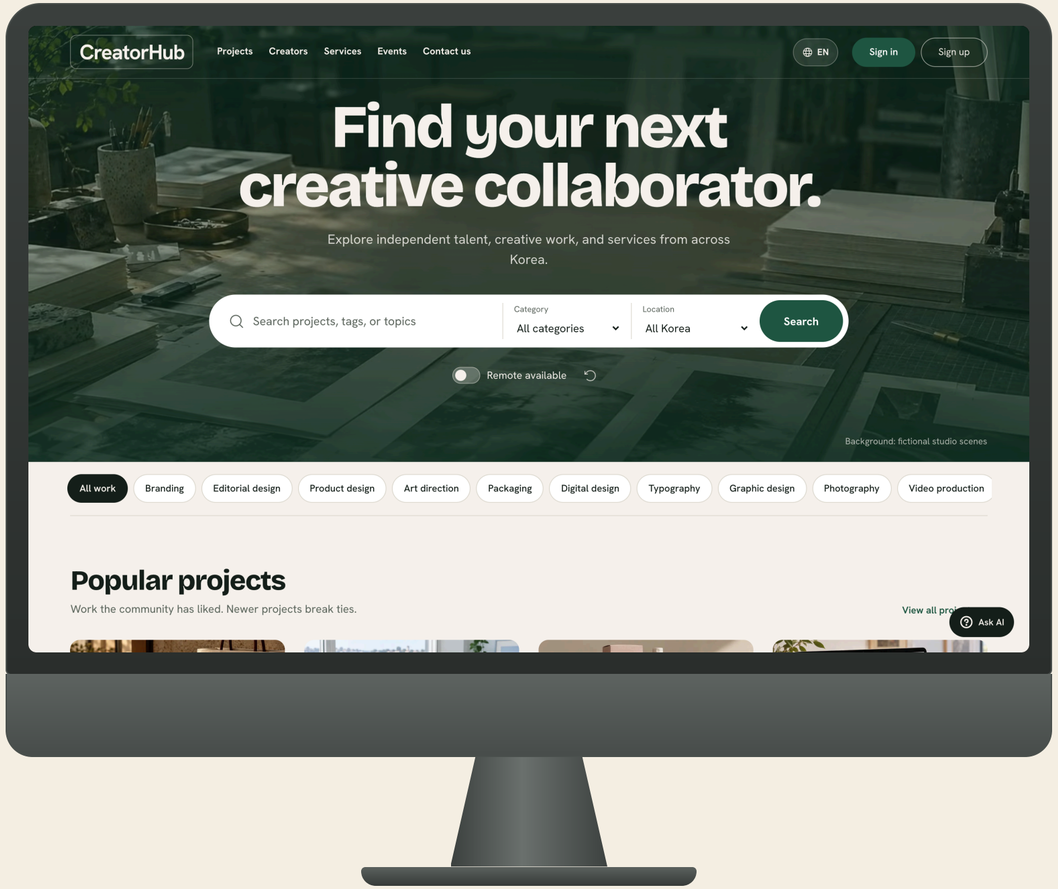
A portfolio-discovery platform where visitors find independent creators across Korea, explore their projects and services, and send private project inquiries.

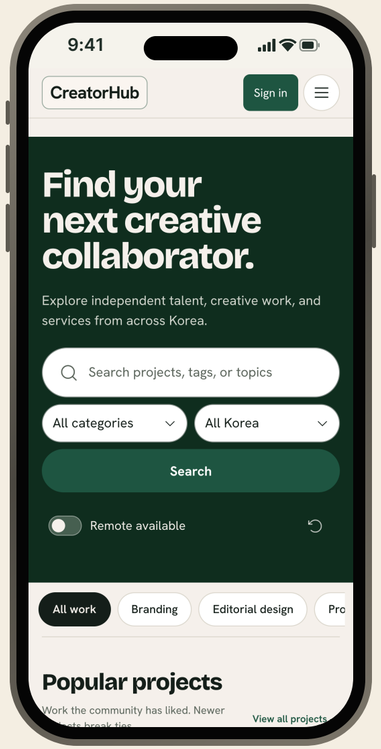
- Search by category, location and remote work
- Creator workspace with admin approval
- Private inquiries with live chat

FRONTEND	Next.js · React · TypeScript
BACKEND	NestJS · GraphQL · Socket.io
DATA	MongoDB · Mongoose
INFRA	Docker · Nginx · GitHub Actions
AI	Gemini API

Visit live site ↗

Repos: CreatorHub · CreatorHub-next





— CREATORHUB · CASE STUDY

One click, one inquiry: retry-safe requests.

Problem. A double-click or a dropped connection could send the same brief or chat message twice. The server saw two new requests and would save both, with two notifications for the creator.

1

Key per form

The browser creates one key for the form and reuses it on every retry

`crypto.randomUUID()`

→

2

Check first

Inside a transaction, a known key returns the record that is already saved

`findOne(clientId, key)`

→

3

Index as a backstop

Two requests at the same moment: the unique index rejects the second insert

`partial unique index`

→

✓

One record

The losing request answers with the first record, so the client sees one result

`uq_inquiry_request`

Same key, different text

Reusing a message key for different content returns a conflict instead of silently keeping the old message.

Versions on every change

Each status change carries the version it was based on, so a stale screen cannot overwrite a newer state.

Verified

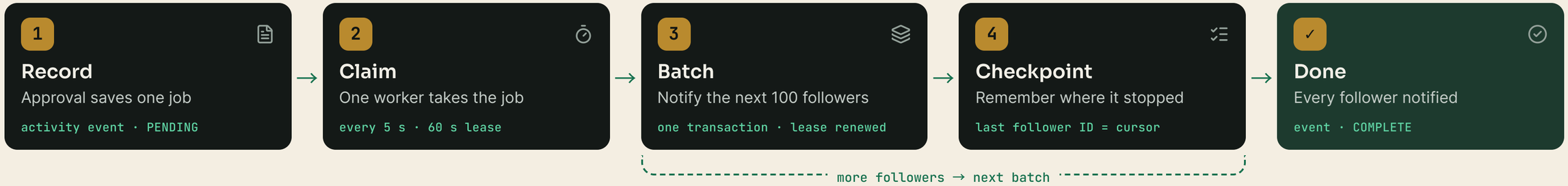
Integration tests submit the same inquiry twice and send parallel messages with one key: exactly one event, one message and one notification.

The same pattern covers likes, saves, follows and comment retries.

— CREATORHUB · CASE STUDY

Follower notifications that survive a worker restart.

Problem. When a creator publishes new work, every follower gets a notification. If the worker crashes halfway through, some followers could be notified twice, or not at all.



Crash or error? An error releases the job for a retry in 30 s; after a crash the 60 s lease simply runs out. Either way, the next claim resumes from the saved cursor.

No duplicates

Each notification is unique per follower and event, so a repeated batch cannot notify anyone twice.

Known limit

Polling MongoDB avoids a separate queue service but adds a few seconds of delay. Retries have no cap yet; next step is to stop after a set number and flag the event for an admin.

Verified

Integration tests cover a rolled-back batch that retries without duplicates, and followers notified only on first publication.

Scope: in-app notifications only, at most one per follower per event. No email or push on this path.

— CREATORHUB · CASE STUDY

Testing, CI/CD and rollback

Both repositories are tested on every push. Releases go through a separate, manually started workflow.

— WHAT IS TESTED

Backend unit · Jest
Business rules: inquiry states, passwords, rate limits

API integration · Jest
Real GraphQL on a throwaway MongoDB replica set

Frontend · Node.js
Sessions, sign-in, the AI chat route, translations

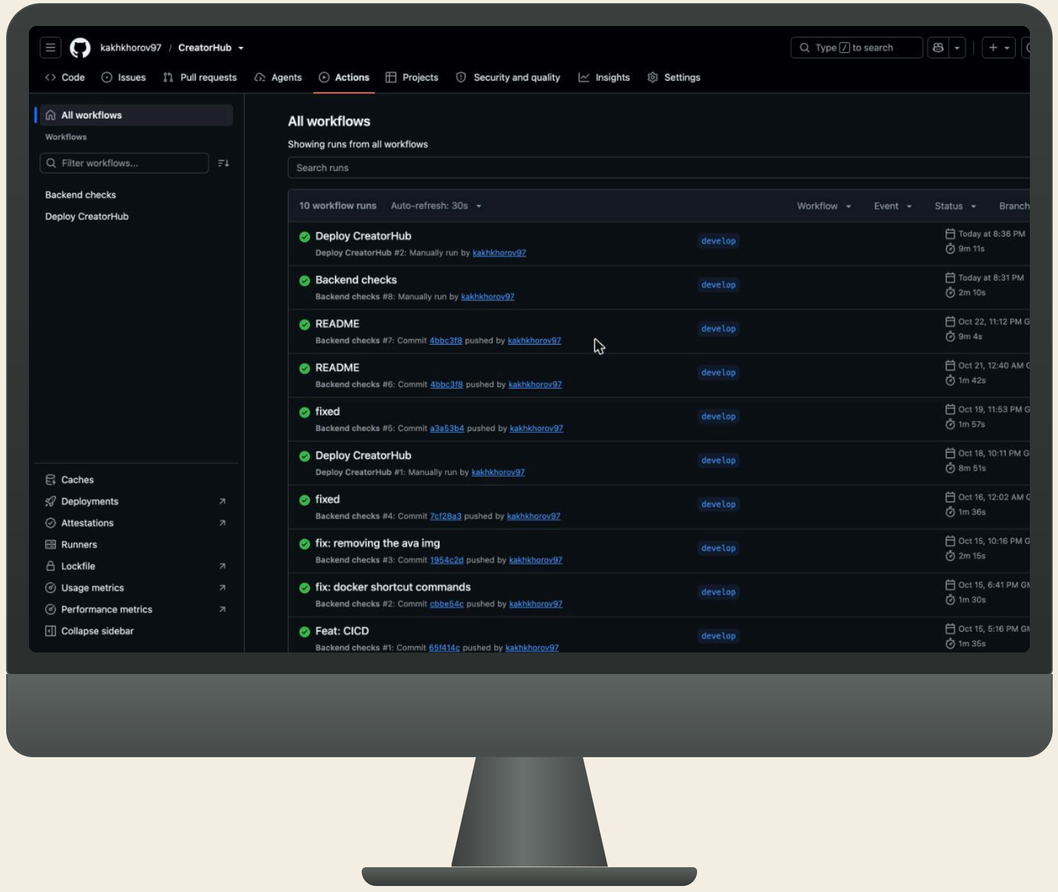
Static checks
TypeScript, ESLint, every query checked against the schema

recorded locally · 17 Oct 2026

```
$ backend   jest           ✓ 162/162 tests · 16 suites
$ backend   deploy scripts ✓ 23/23
$ frontend  node --test    ✓ 108/108
$ both      typecheck · lint · contract (119 ops) ✓ passed
= 293 automated checks recorded (162 + 23 + 108)
```

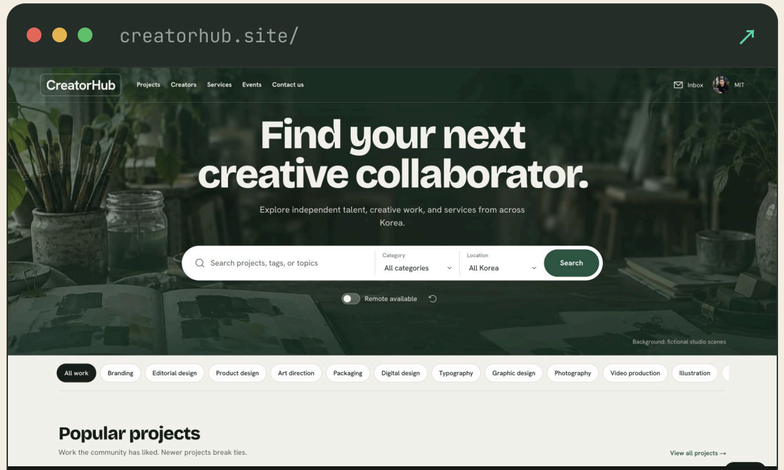
— HOW A RELEASE SHIPS

1. **Every push** to develop or main runs install, typecheck, lint, tests and build.
2. **Deploy workflow,** started by hand, re-runs all checks for one backend + frontend commit pair.
3. **Restricted SSH** to a deploy-only account that accepts nothing but “deploy” and two commit IDs.
4. **Health-checked switch:** new images get a configured window of up to 180 s to report healthy; on failure the previous images return automatically.

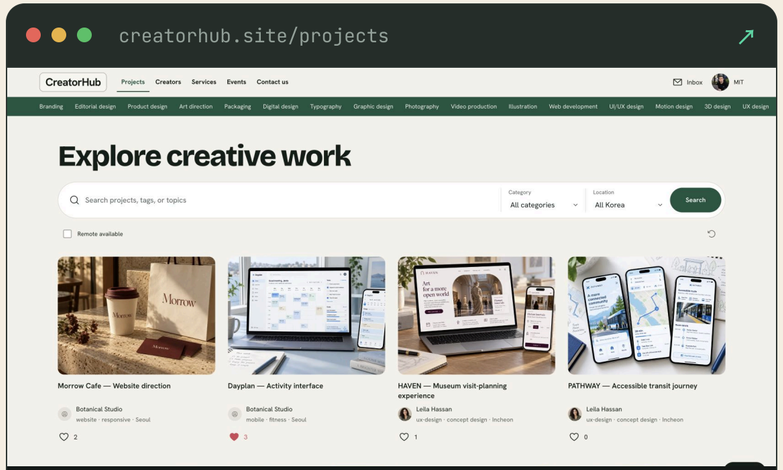


Screen: GitHub Actions on 24 Oct 2026, 10 of 10 runs passing, including two manual Deploy CreatorHub runs. 180 s is a configured limit, not a measured recovery time. Database writes are never rolled back.

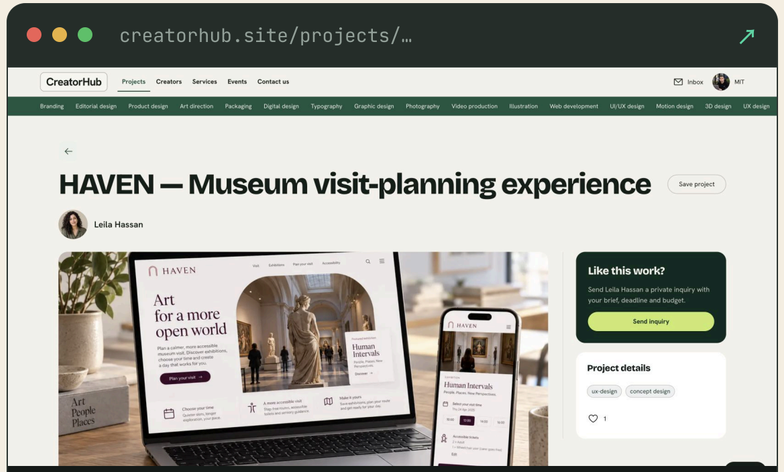
The live product.



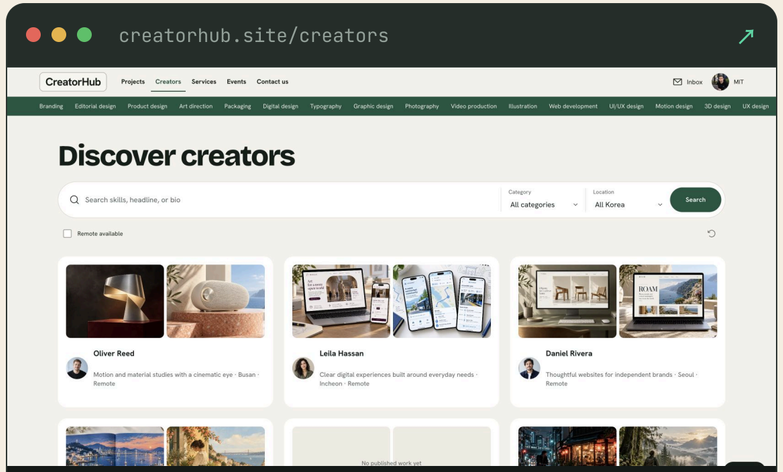
Live homepage with search and project discovery.



Project discovery with category, location and remote-work filters.



Project page with fictional demo content and AI-generated concept artwork.



Creator directory populated with demonstration profiles and work.

— WHAT IT DOES

- One MongoDB aggregation filters, sorts and pages results; search state lives in the URL
- Profiles, services with fixed or custom prices, and admin-managed events
- Like, save, comment and follow, one per person and item
- Creator workspace: edits stay hidden until an admin approves them
- Admin area for users, roles, moderation and site copy, behind a server-side role guard
- Sign-in with email, Google, Kakao or Telegram; EN, KO and UZ

Captured on creatorhub.site, 22 Oct 2026. Creators, projects and artwork are fictional demo content.

— CREATORHUB · WORKFLOW

Private inquiries and live chat

Clients send a creator a brief with budget and deadline, then talk it through in a private chat.

Draft
Client writes the brief

→

Sent
Creator can decline

→

In discussion
After the first reply

→

Closed
Agreed, not proceeding, or no reply in 7 days

Private by design

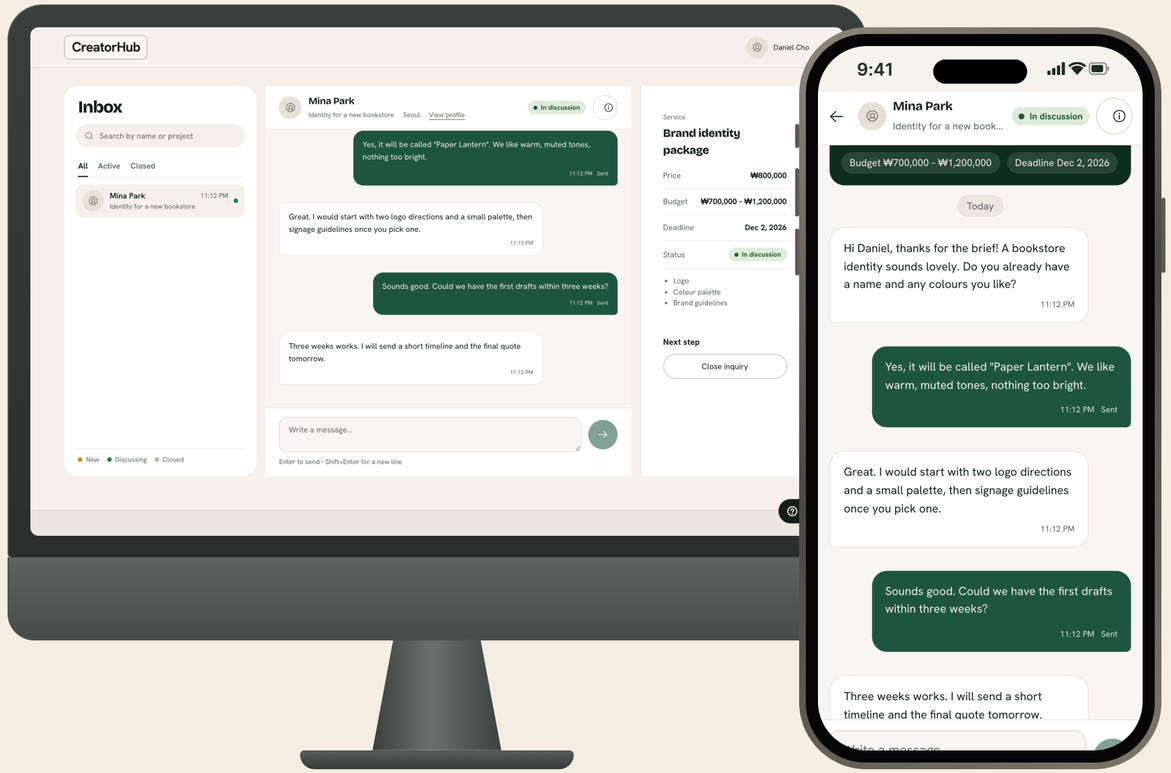
Only the client and the creator can open an inquiry. Creators never see drafts.

No duplicates

A double-click or network retry never creates a second inquiry or message.

Live chat

Socket.io rooms use the same access check. The database stays the source of truth.



Captured from the current app running locally, with fictional accounts and messages.

— CREATORHUB · CASE STUDY

Keeping the AI finder fast, affordable and safe.

Problem. The AI finder is open to every visitor, and each answer is a paid Gemini call. Without limits, one script could flood it, slow lookups would keep visitors waiting, and a busy model would show errors.

1

Gate the route

Browser origin check, messages up to 1,500 characters, 8 per minute per visitor

`origin · size · rate`

→

2

Daily cap

A daily limit bounds the model bill even under abuse

`daily cap · 200 by default`

→

3

Time-boxed retrieval

Up to 3 help entries plus live public data, and the lookup gives up after 3.5 s

`timeout 3.5 s`

→

✓

Graceful fallback

A busy model gets one retry on a fallback model; without a key, visitors are sent to Help

`429 / 500 / 503 → fallback`

Grounded answers

Low temperature, up to 8 previous turns, and a prompt that tells the model to treat sources as data, not instructions.

Key stays private

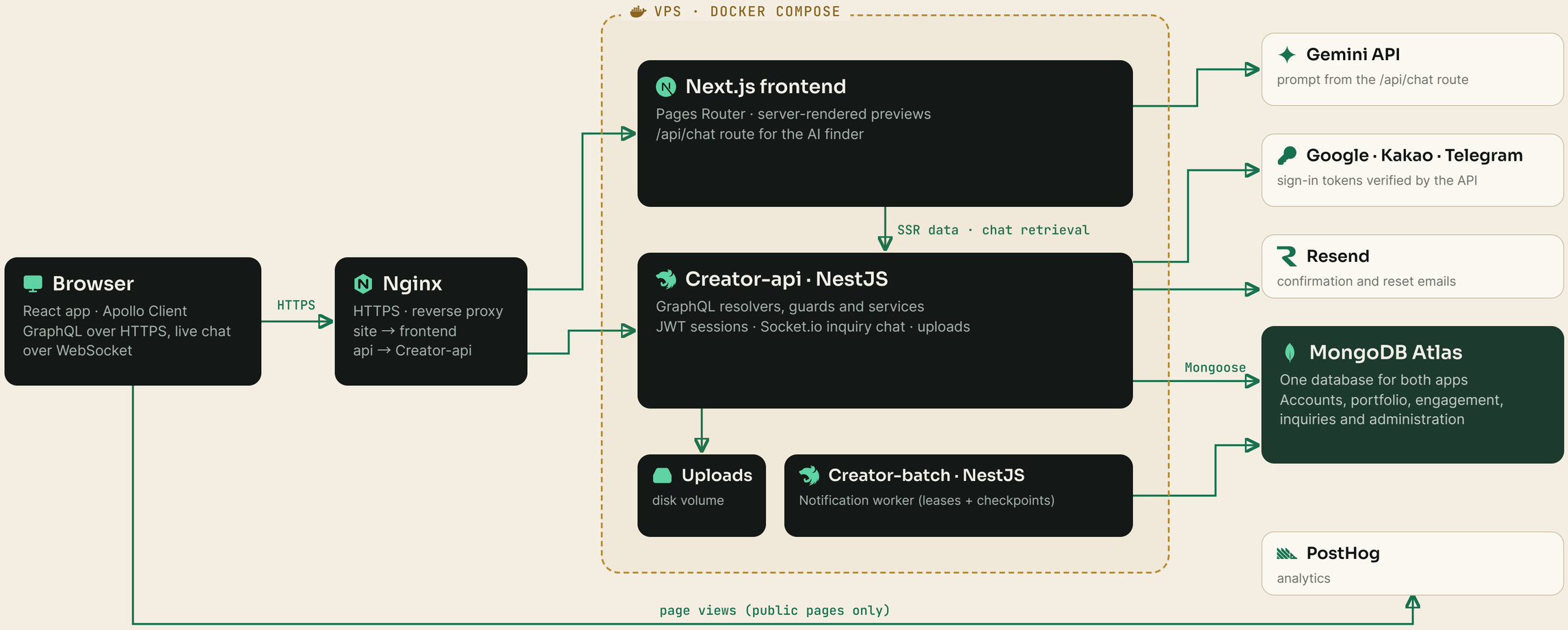
The Gemini key lives only in the server route. The browser never sees it.

Known limit

Retrieval is keyword matching plus one GraphQL query, with no embeddings yet. Next step: semantic search.

— CREATORHUB · ARCHITECTURE

Three applications, one data model.



— CREATORHUB · DATA MODEL

How the data is designed.

21 collections in one MongoDB database, shared by the API and the worker. References are checked in the services, and unique indexes enforce the rules that must never break.

Accounts

`users` `authIdentities` `authSessions` `authTokens`

- Each sign-in method is its own record, so one person can use email, Google and Kakao
- Refresh and email tokens are stored only as hashes and expire automatically
- Sessions are records, so suspending a user ends all of them

Portfolio

`creatorProfiles` `categories` `portfolioProjects` `services`

- Readable, unique slugs for profiles and for projects within a creator
- Social links and project media are embedded, because they are always read with their parent
- Indexes follow the public feeds: status, then newest first

Engagement

`creatorFollows` `projectLikes` `savedProjects` `projectComments` `activityEvents`

- One like, save or follow per person and item, enforced by unique indexes
- Unfollowing switches a follow off instead of deleting it
- Activity events carry the worker's lease and cursor for safe delivery

Inquiries

`inquiries` `inquiryMessages` `inquiryEvents` `serviceFeedback`

- Service terms are copied into the inquiry as an embedded snapshot
- Retry keys with partial unique indexes stop duplicate inquiries and messages
- Every status change is a numbered event; one review per completed inquiry

Administration

`contentReviews` `notifications` `creativeEvents` `siteContent`

- At most one pending review per profile, project or service
- A notification is unique per recipient and event
- Site copy is one keyed document, so edits need no redeploy

Inquiry fields are enlarged on page 16. The full ER diagram is a separate high-resolution file (CreatorHub-ER-all-collections-expanded-HQ.png).

Inquiries, field by field

PK primary key FK → referenced collection ⬚ embedded schema

inquiries

PK	_id	objectId
	clientRequestKey	string
FK	clientId	→ users
FK	creatorId	→ creatorProfiles
FK	serviceId	→ services
	subject	string
	brief	string
	budgetMin	int
	budgetMax	int
	currency	enum
	desiredDeadline	date
	serviceSnapshot	embedded
	status	enum
	outcome	enum
FK	closedById	→ users
	closedAt	date
	submittedAt	date
	lastMessageAt	date
	version	int
	createdAt	date
	updatedAt	date

InquiryServiceSnapshot

embedded in inquiries.serviceSnapshot

	title	string
	pricingType	enum
	startingPrice	int
	currency	enum
	estimatedDays	int
	deliverables	string[]

inquiryMessages

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	senderId	→ users
	clientMessageKey	string
	body	string
	readAt	date
	createdAt	date

inquiryEvents

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	performedById	→ users
	type	enum
	fromStatus	enum
	toStatus	enum
FK	messageId	→ inquiryMessages
	version	int
	createdAt	date

serviceFeedback

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	clientId	→ users
FK	creatorId	→ creatorProfiles
FK	serviceId	→ services
	rating	int
	feedback	string
	createdAt	date
	updatedAt	date

— SELECTED PROJECT · 02 · LIVE

CargoCheck AI.

Live at cargocheck-ai-web.vercel.app with Gemini extraction in production. An early build: no customs certification and no automatic approval.

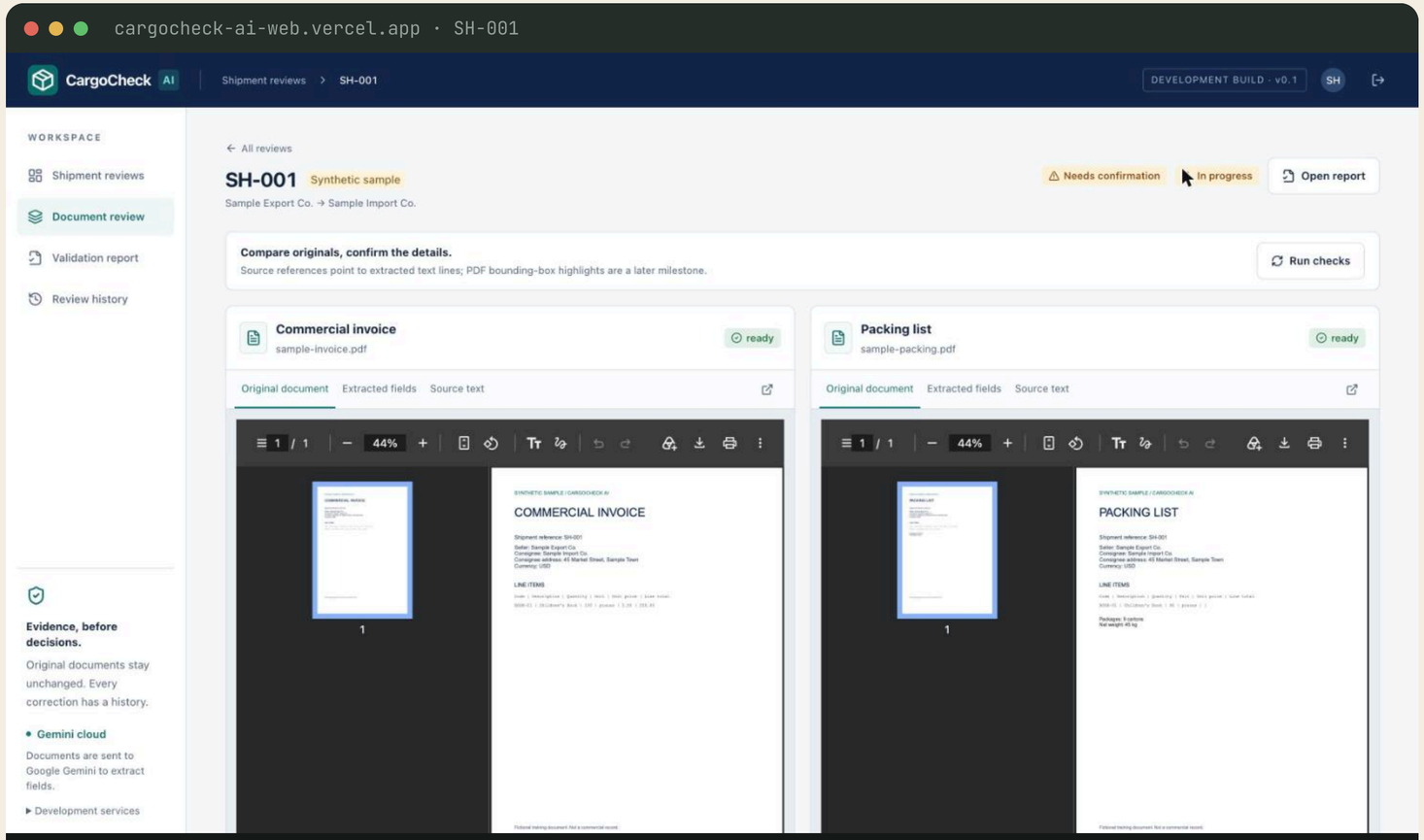
A review app for commercial invoices and packing lists. Gemini extracts the fields, a reviewer confirms each value against the original, explicit rules flag mismatches, and every decision is recorded.

- Upload both documents; Gemini extracts the fields
- Confirm each value against its source line
- Rule checks, one decision per flagged item
- Audit history and a PDF review report

FRONTEND	React · TypeScript · Vite
BACKEND	NestJS · Zod · decimal.js
AI	Gemini API · Tesseract · Poppler
DATA	Neon · Upstash · BullMQ
INFRA	Vercel · Render · Docker

Open the live app ↗

Repo: cargocheck-ai



Live app, 24 Oct 2026: both original documents processed in Gemini cloud mode (synthetic sample).

— CARGOCHECK AI · EXTRACTION

From upload to source-linked fields.

Problem. Reviewers retype and cross-check invoice and packing-list values by hand. A model can misread a value, so every value must stay traceable to its source and be confirmed by a person.

1 — Upload
Invoice and packing list; file signatures checked; originals never rewritten

`PDF · JPG · PNG · 10 MB`

→

2 — Extract
In production, Gemini structured output reads both documents

`gemini-3.5-flash-lite`

→

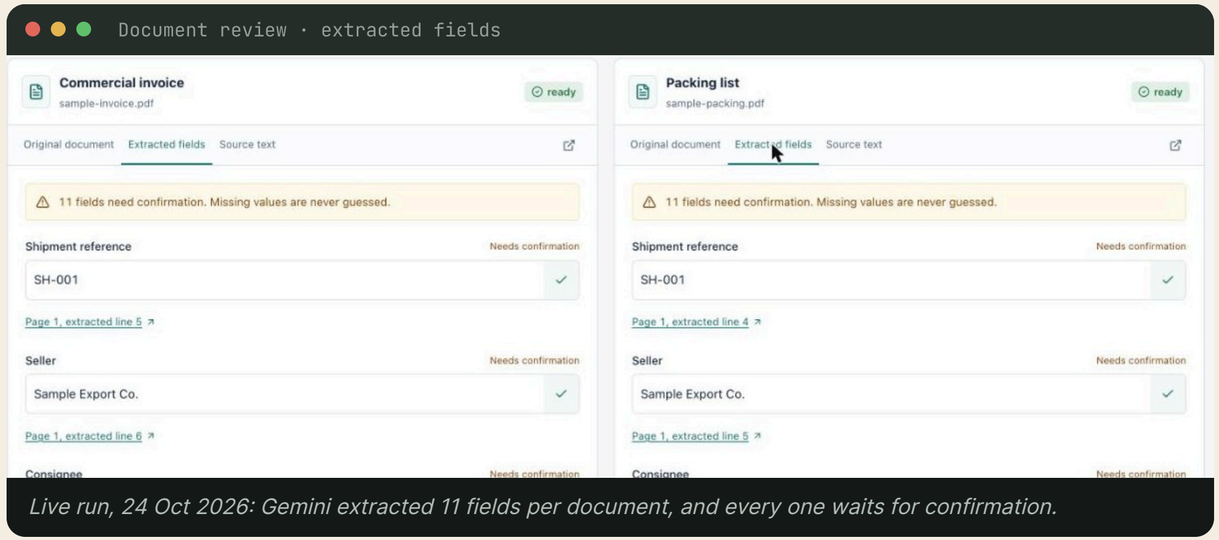
3 — Validate
The response is parsed against a Zod schema; unknown values stay empty

`missing values stay null`

→

✓ — Confirm
Each value keeps its page and line; the reviewer confirms or corrects it

`page 1 · extracted line 13`



Live run, 24 Oct 2026: Gemini extracted 11 fields per document, and every one waits for confirmation.

Never guessed

A value the model cannot find stays empty. The banner says so: missing values are never guessed.

Two modes

Production uses Gemini. A local mode runs English Tesseract OCR and a labelled-template parser with no AI key.

Untrusted input

Document text is treated as data, not instructions, and the API key stays on the server.

Explicit rules, human decisions.

Once both documents are confirmed, rules compare them. A rule only flags a difference; the reviewer decides what it means and writes down why.

— WHAT IS COMPARED

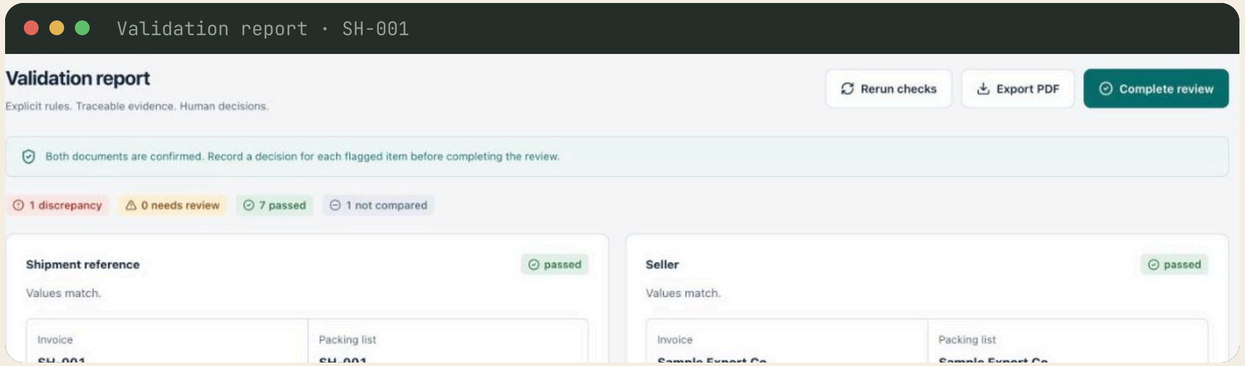
- Shipment reference, seller, consignee and address
- Explicit product codes; ambiguous matches stay review cases
- Quantities in compatible units; incompatible units are flagged, never converted
- Invoice arithmetic in decimal.js: quantity × unit price = stated total

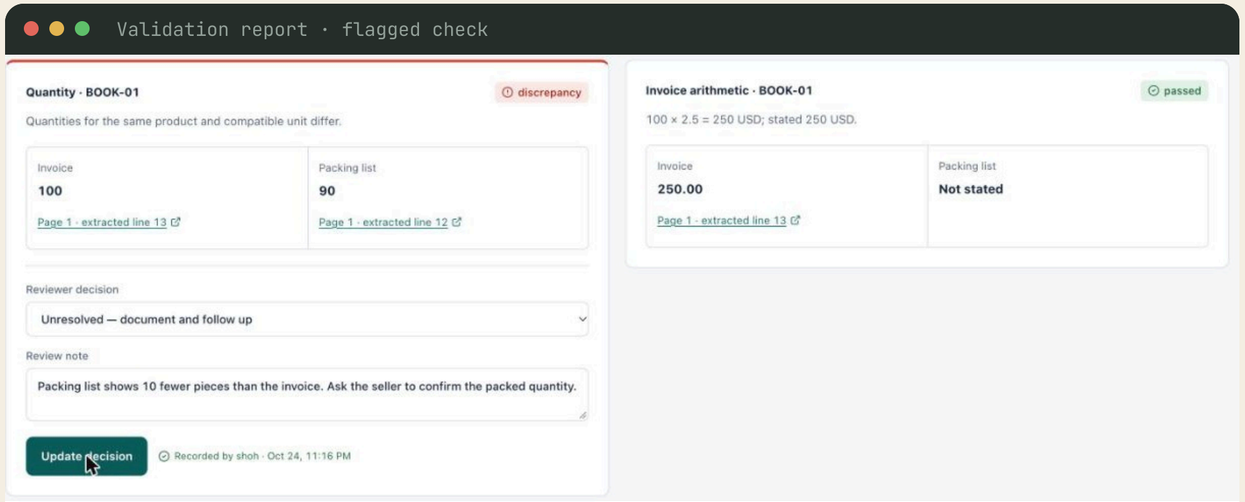
One decision per flag

The review cannot be completed until every flagged check has a recorded decision and note.

Not an approval

Completing a review records the reviewer's work. It does not approve a shipment.





Live, 24 Oct 2026: 100 invoiced vs 90 packed pieces of the same product, with the reviewer's recorded decision. Synthetic sample values.

How it runs in production.

Browser
React app; /api requests go to the same origin

→

Vercel
Static React/Vite build; rewrites /api/* to the API

→

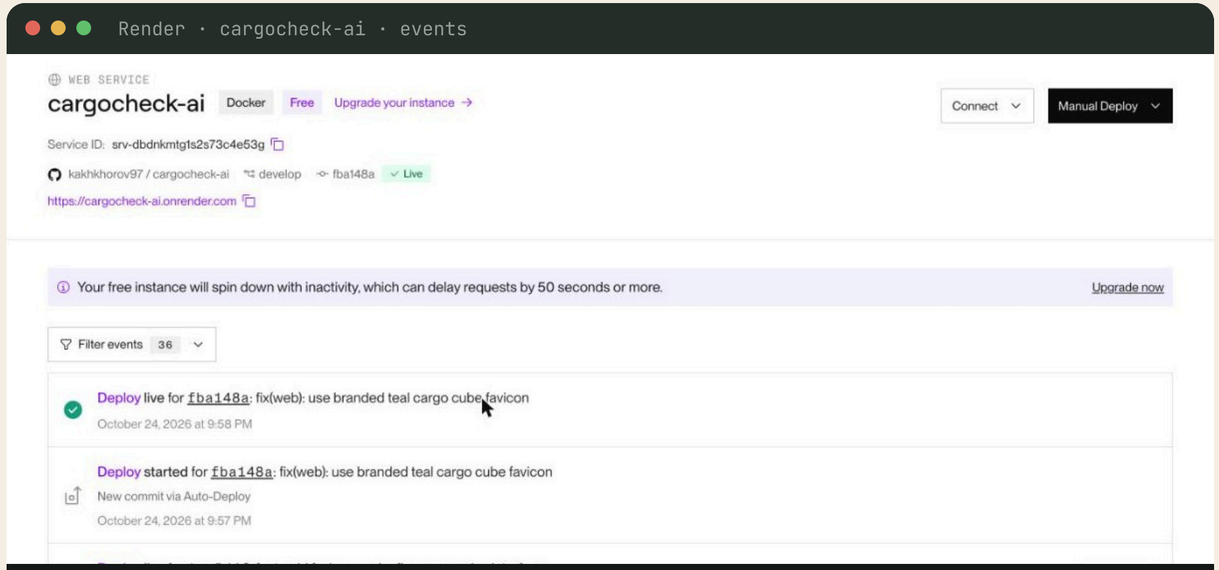
Render
NestJS API in Docker; auto-deploys from the develop branch

→

Neon PostgreSQL
Application state, one transaction-locked JSONB row

Upstash Redis
BullMQ extraction jobs, up to 3 attempts

Gemini API
Structured extraction from the documents



Render events, 24 Oct 2026: each commit to develop builds the Docker image and goes live automatically.

Secrets stay server-side
Database URL, Redis URL and Gemini key live in the Render environment, never in the browser or the repository.

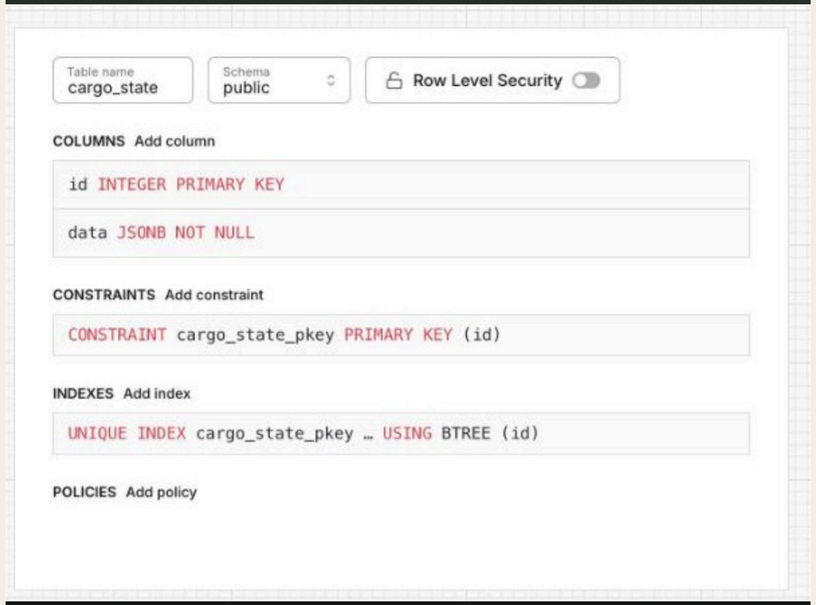
Production guard
With NODE_ENV=production the API refuses to start without a valid, non-local PostgreSQL URL.

Free-tier limits
The free Render instance sleeps when idle, so the first request can take up to about 50 s.

— CARGOCHECK AI · DATA LAYER

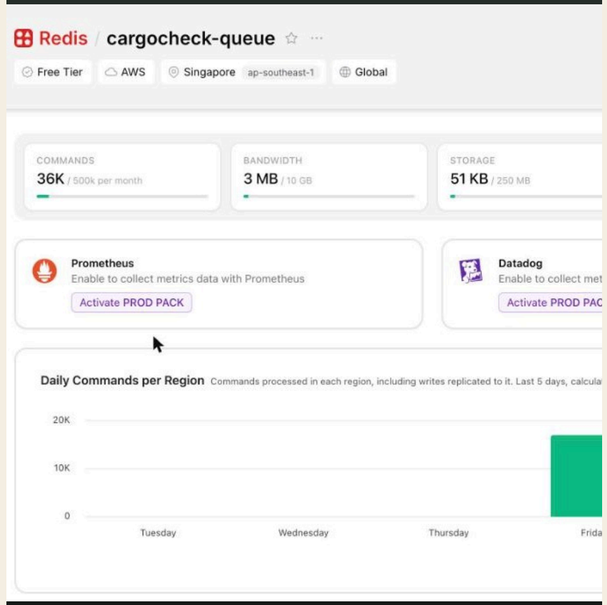
Neon PostgreSQL and Upstash Redis.

Neon · CargoCheckAI · production · cargo_state



The production table in Neon: id INTEGER PRIMARY KEY, data JSONB NOT NULL.

Upstash · cargocheck-queue



The BullMQ queue on Upstash. Command counts include idle queue polling, not only processed jobs.

— EVERY CHANGE, IN ONE TRANSACTION

1. **BEGIN**
2. **SELECT data … FOR UPDATE** locks the state row
3. **Apply the change** in application code
4. **UPDATE … COMMIT,** or **ROLLBACK** on any error

Why one row

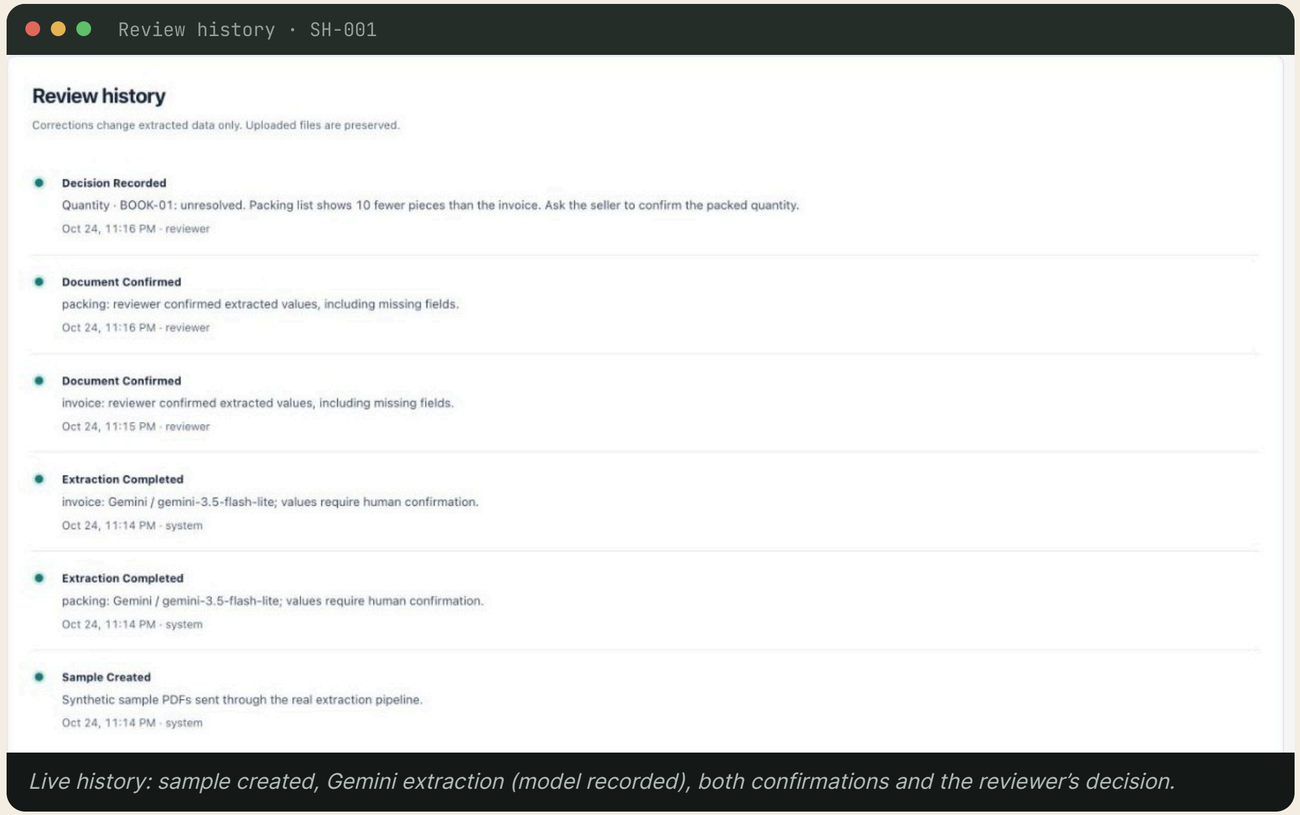
Every write is consistent with no migrations, which suits a first single-instance release.

The trade-off

Each write rewrites the whole state and waits for one lock. It will not scale to many users.

Next, not built yet: normalised tables for shipments, documents, fields, checks and decisions, with migrations and indexes.

History, report and what is verified



Live history: sample created, Gemini extraction (model recorded), both confirmations and the reviewer's decision.

Corrections reopen work

A correction keeps the original value, marks earlier decisions as invalid, and can reopen a completed review. Stale edits get a version conflict.

Accounts and uploads

Per-account ownership checks, HTTP-only cookie sessions, salted scrypt hashes, upload signature checks, rate limits and same-origin checks.

Verified

- ✓ Live run on 24 Oct 2026: Gemini extraction, confirmation, checks and a recorded decision
- ✓ Neon PostgreSQL and Upstash Redis in production
- ✓ 11 automated tests in the repository

Not yet

- ◌ Accuracy, speed or cost measurements
- ◌ Normalised schema and object storage
- ◌ Korean UI and Korean OCR

— SELECTED PROJECT · 03 · LIVE

Lexara.

Live, collaborative project; I am a contributor. I helped another developer build Lexara; the code was pushed through my collaborator's GitHub account, and my repository is a fork of MakhmudSD/Lexara.

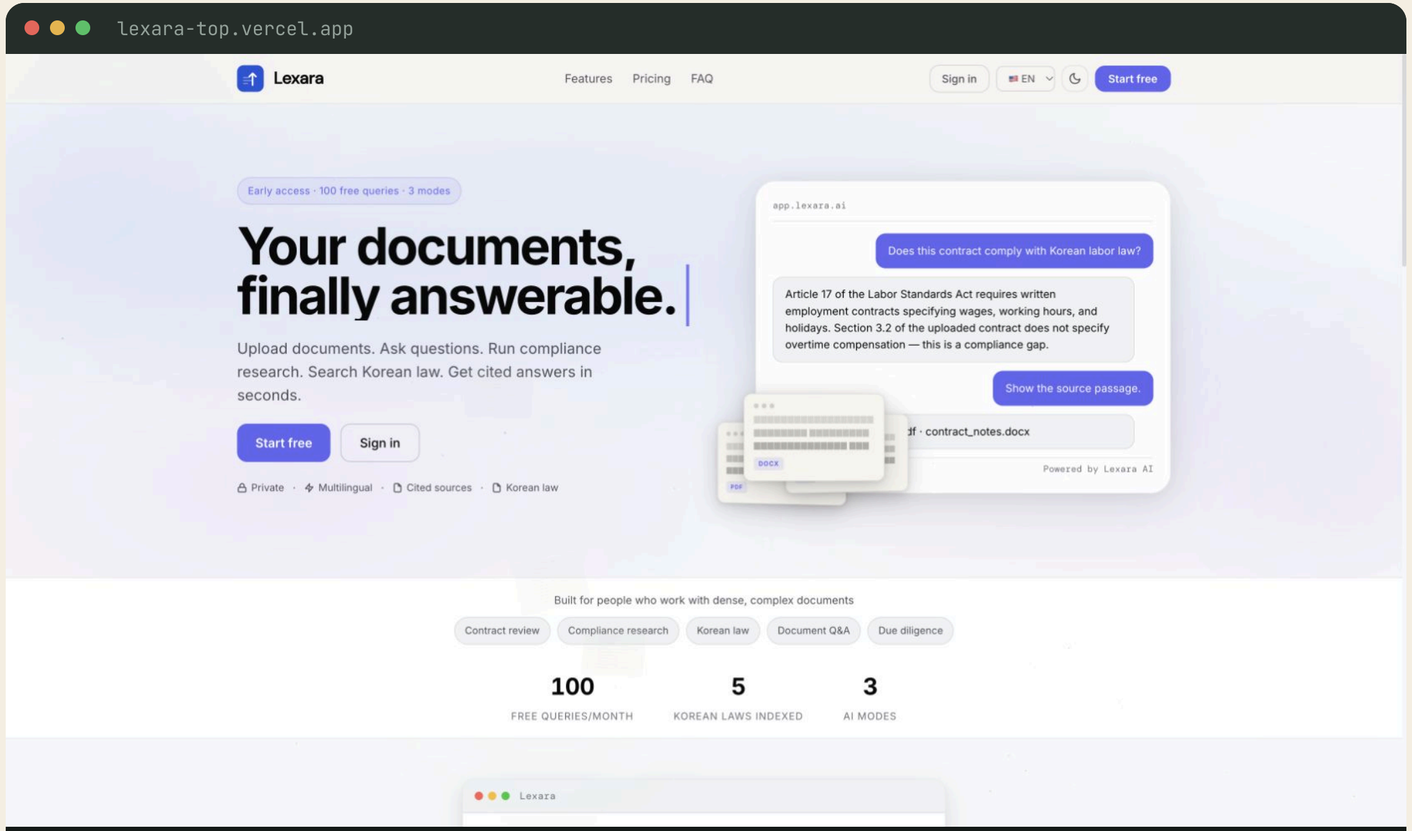
A document Q&A platform: uploaded documents are organised into workspaces, and answers cite the passages they come from.

— PRODUCT ARCHITECTURE (TEAM-BUILT, CURRENT SOURCE)

- FastAPI with PostgreSQL, SQLAlchemy and Alembic
- pgvector similarity plus BM25, merged with RRF
- Uploads return 202 and are processed as background tasks, with status polling
- React/Vite frontend; OpenAI models; optional R2 storage

Public site ↗

Repo: **kakhkhorov97/Lexara** (fork of **MakhmudSD/Lexara**)



Lexara public landing page, 22 Oct 2026. The answers in the embedded preview are demonstration content.

The older README describes FAISS; the inspected source uses pgvector. Sign-in, upload and chat were not tested for this portfolio.

— SELECTED PROJECT · 04 · READY TO DEPLOY

Korea Visa Assistant

Ready to deploy: the Docker image passes its CI smoke test and a Render Blueprint is prepared. No public URL yet.

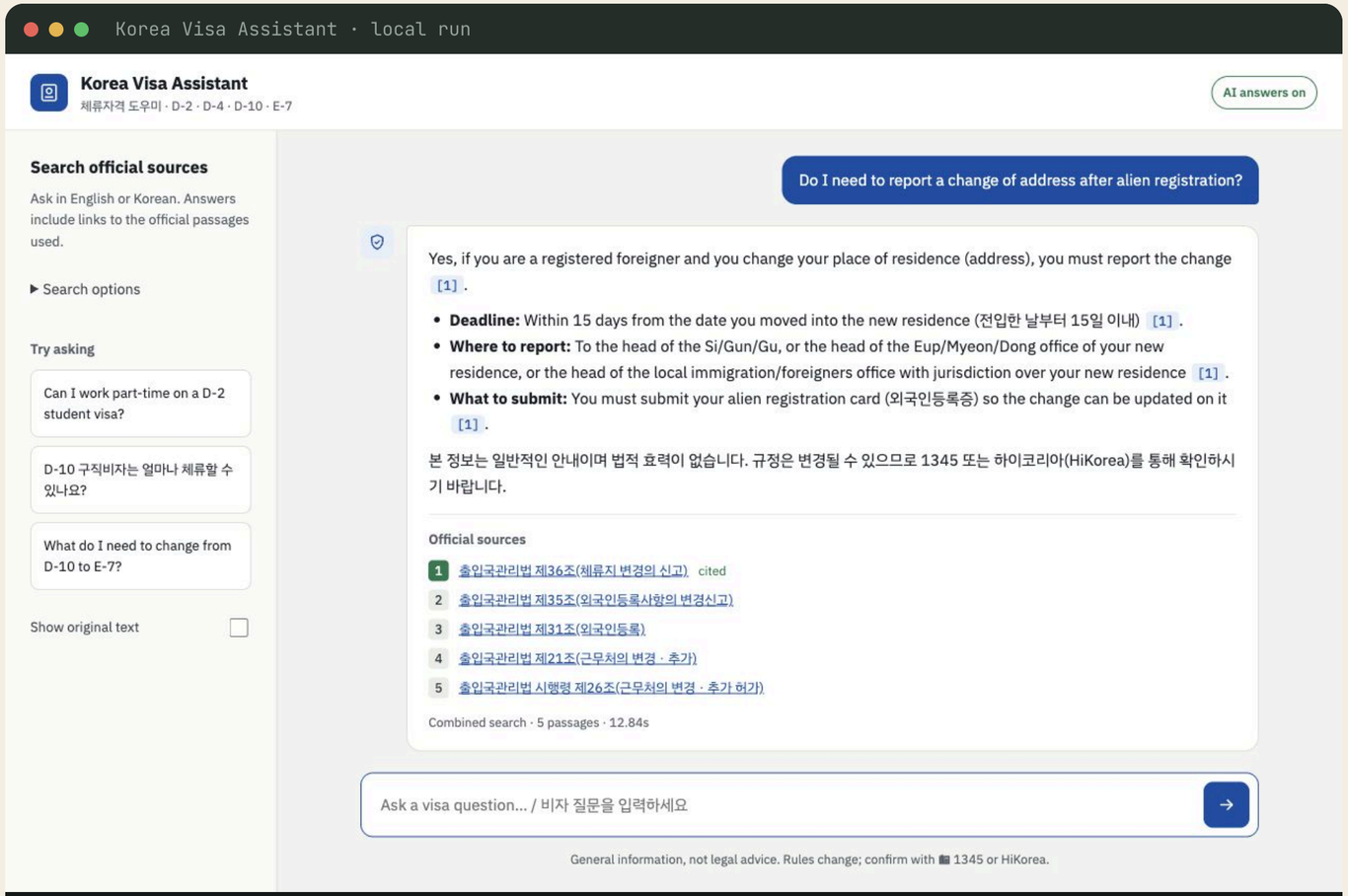
Answers questions about Korean student and job-seeker visas (D-2, D-4, D-10, E-7) in English or Korean, citing the official article behind every claim.

— MEASURED RETRIEVAL · 11 LABELLED QUESTIONS · 22 STATUTE CHUNKS

	hit@5	recall@5	MRR
BM25	0.818	0.636	0.818
Vector	1.000	0.939	0.848
Hybrid (RRF)	**1.000**	**0.909**	**0.939**

Measured 24 Oct 2026. A small first set, not independently annotated; it measures retrieval, not legal answer accuracy.

Repo: korea-visa-rag



Local run with a real Gemini answer and official citations, from the project repository (24 Oct 2026). Not a public deployment; general information, not legal advice.

— CREATORHUB · CASE STUDY

Retrieval built for Korean legal text.

Problem. Korean law is structured as article › paragraph › item, and questions use exact codes like D-10 or E-7-1. Plain fixed-size chunks and generic tokenizers split both apart.

1

Chunk by article

One chunk per 제N조 or 제N조의M, keeping its paragraphs and items

`article label + source URL`

→

2

Tokenize

Kiwi morphological analysis; visa codes extracted first as single tokens

`D-10 · E-7-1`

→

3

Retrieve twice

BM25 for exact terms, Chroma with Gemini embeddings for meaning

`gemini-embedding-001`

→

✓

Fuse and cite

RRF merges the two lists; answers cite the passages used

`RRF_K`

Cited answers

The prompt allows only the retrieved sources and asks the model to say when they do not answer. That is a design choice, not proof of correctness.

One engine, two interfaces

FastAPI web and the Telegram bot call the same assistant, including the no-key search-only mode.

Why hybrid

On the labelled set, fusion lifted MRR from 0.818 (BM25) and 0.848 (vector) to 0.939, so the right article ranks first more often.

Python · FastAPI · Kiwi · rank_bm25 · Chroma · Gemini API · Telegram Bot API · Docker · pytest · GitHub Actions

— EARLIER PROJECT · READY TO DEPLOY

Sofiya Bookstore.

A React/Redux Toolkit storefront on an Express REST API with MongoDB: readers browse, fill a basket and follow their orders, and an EJS admin panel manages the catalogue. Every order query is filtered by the signed-in member. Repos: Sofiya-react · Sofiya-Bookstore

Basket
Books and quantities chosen

→

Order saved
Line items stored; delivery is free from 100, otherwise 5

→

Paused
Waiting for the reader to confirm

→

In process
Confirmed; the member earns a point

→

Delivered
Order complete

Admin panel

Server-rendered EJS pages: sign-in, the product list, new products with up to 5 images, product edits, and a user list where the admin can change a member's status.

Two sign-in paths

Readers get a JWT in a cookie for 24 hours, checked on every API call. The admin panel uses sessions stored in MongoDB for 6 hours, and only store accounts pass its guard.

What I would change now

Item prices come from the browser and the payment step is a mock-up. Today I would recompute totals on the server and connect a real payment provider.

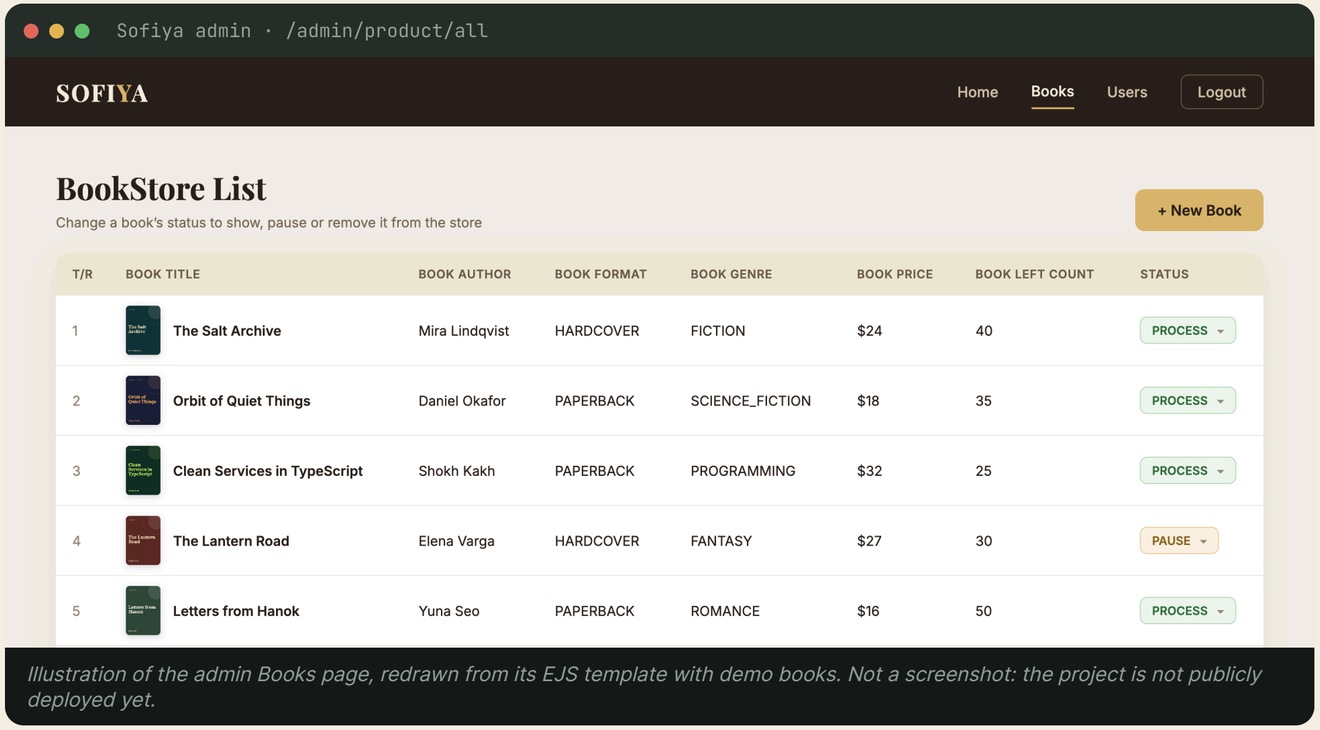


Illustration of the admin Books page, redrawn from its EJS template with demo books. Not a screenshot: the project is not publicly deployed yet.

— CONTACT

From idea to production

EMAIL	kakhkhorov97@gmail.com
PHONE	010-7901-7555
KAKAOTALK	kakhkhorov_A
GITHUB	github.com/kakhkhorov97
PORTFOLIO	sean-kakhkhorov.site
CREATORHUB	creatorhub.site
CARGOCHECK AI	cargocheck-ai-web.vercel.app
ADDRESS	Suwon, Gyeonggi-do, South Korea

Portfolio
sean-kakhkhorov.site

CreatorHub
creatorhub.site

CargoCheck AI
vercel.app

